

Тема № 14. Потокковая многозадачность

Дополнительная литература:

Уильямс Э. Параллельное программирование на C++ в действии. Практика разработки многопоточных программ.- М.: ДМК Пресс, 2016. – 672 с.

14.1. Понятие многозадачности

Два вида многозадачности:

- многозадачность, основанная на процессах;
- многозадачность, основанная на потоках.

Процесс можно определить как копию (экземпляр) выполняющейся программы. В данном случае копия – понятие статическое. Т.е. процесс– это объект, который не выполняется, а просто «владеет» выделенным ему адресным пространством, другими словами, процесс – структура в памяти.

В адресном пространстве процесса находятся не только код и данные, но и потоки (thread) – выполняющиеся объекты. При запуске процесса автоматически запускается поток (он называется главным). Главный поток может запускать другие «дочерние» потоки.

Потоки могут работать параллельно (одновременно) друг с другом с многопроцессорных системах (в однопроцессорных системах работают «как бы параллельно» за счет временного разделения) с учетом их приоритетов и имеют доступ к ресурсам процесса (приложения).

Понятие потока (thread) как части процесса не следует путать с понятием потока ввода-вывода (stream).

Потоки бывают интерфейсные (способны принимать и обрабатывать сообщения) и рабочие.

Потоки можно создавать с помощью, например, API функций Windows (платформенно-зависимые средства) или с помощью стандартной библиотеки классов Си++ (платформенно-независимые средства).

Поддержка многопоточности в языке Си++ определена стандартом 2011 г., до этого приходилось использовать платформенно-зависимые средства.

14.2. Многозадачность в стандартной библиотеке C++. Высокоуровневый интерфейс

Используется функция `std::async()` и шаблон класса `std::future<>`

Требуется вычислить:

`func1()+func2()`

Обеспечить параллельное вычисление.

Пример:

```
#include <iostream>
#include <future>
using namespace std;
// Требуется вычислить :
// func1() + func2()
// Обеспечить параллельное вычисление.

int func1()
{
```

```

        for (int i = 0; i < 5; i++)
        {
            cout << "func1 i=" << i << endl;
        }
        return 5;
    }
    int func2()
    {
        for (int i = 0; i < 10; i++)
        {
            cout << "func2 i=" << i << endl;
        }
        return 10;
    }

    class X
    {
    public:
        void print(int n)
        {
            for (int i = 0; i < 10; i++)
                cout << "X n=" << n++ << endl;
        }
    };

    void main()
    {
        future<int> rez1(async(func1)); // Асинхронное выполнение сейчас или позже
        int rez2 = func2(); // Запускаем синхронное выполнение
        int Rez = rez1.get() + rez2; // Вызов get - ожидание завершения func1
        cout << "Rez=" << Rez<<endl;
        // Передача в async указатель на функцию класса
        X x;
        future<void> a(async(&X::print, x, 11)); // Пытаемся вызвать x.print(11);
        a.get(); // Ждем завершения
    }
}

```

Ниже представлен пример многопоточной программы, в которой умножается матрица на матрицу, потоки создаются с помощью класса thread. В этой же программе вычисляется скалярное произведение двух векторов, потоки создаются с помощью шаблона future. Потоки при создании помещаются в контейнер шаблона vector.

```

#include <iostream>
#include <thread>
#include <vector>
#include <future>
using namespace std;
//vector<vector<int>> Matr; <n x m> * <m x k> -> <n x k>
// Потоквая функция: умножает строку 1-ой матрицы на столбец 2-ой матрицы, результат записывается
в параметр Rez
void str_in_col(vector<int> & str, // Строка 1-ой матрицы
vector<vector<int>>& Matr, // 2-ая матрица целиком
int j_col, // Номер столбца, на который умножаем
int& rez) // Результат
{
    rez = 0;
    for (int i = 0; i < str.size(); ++i)
        rez += str[i] * Matr[i][j_col];
}

```

```

// Потокоская функция для умножения двух чисел
int proiz(int a, int b)
{
    return a * b;
}

int main()
{
    int n = 3, m = 4, k = 5;
    // Скалярное произведение двух векторов через future
    vector<int> V1{ 1, 2, 3, 4 }, V2 = { 1, 2, 3, 4 };
    vector<future<int>> VF; // Вектор future
    // Создаем потоки
    for (int i = 0; i < V1.size(); i++) VF.push_back(async(proiz, V1[i], V2[i]));
    int Rez = 0;
    // Получаем результаты
    for (auto & pos : VF) Rez += pos.get();
    cout << "Rez=" << Rez << endl;

    // Далее произведение матриц через thread
    vector<vector<int>> Matr1(n), Matr2(m), MatrRez(n);
    cout << "Matr1\n";
    for (int i = 0; i < n; i++)
    {
        Matr1[i].resize(m);
        MatrRez[i].resize(k);
        for (int j = 0; j < m; j++)
        {
            Matr1[i][j] = rand() % 10;
            cout << Matr1[i][j] << ' ';
        }
        cout << endl;
    }
    cout << "Matr2\n";
    for (int i = 0; i < m; i++)
    {
        Matr2[i].resize(k);
        for (int j = 0; j < k; j++)
        {
            Matr2[i][j] = rand() % 10;
            cout << Matr2[i][j] << ' ';
        }
        cout << endl;
    }
    vector<thread> Th; // Вектор для хранения потоков
    for (int i = 0; i < n; i++)
    {
        // Создаем потоки
        for (int j = 0; j < k; j++)
            Th.push_back(thread(str_in_col, ref(Mat1[i]), ref(Mat2), j,
ref(MatRez[i][j]))));
    }
    // Ждем завершения потоков
    for (auto & pos : Th)
        // if (pos.joinable())
        pos.join();
    cout << "MatrRez\n";
    for (auto & pos: MatrRez)

```

```

    {
        for (auto pos2 : pos)
            cout << pos2 << " ";
        cout << endl;
    }
}

```

14.3. Многозадачность в стандартной библиотеке C++. Низкоуровневый интерфейс

Для любого потока, включая главный, в заголовочном файле `<thread>` объявлено пространство имен `std::this_thread`, в котором предусмотрены потоковые глобальные функции: `this_thread::get_id()`; // Получить идентиф. Потока
`this_thread::sleep_for(dur)`; // Блокировать поток на период
`this_thread::sleep_for(chrono::seconds(10))`; // Пример
`this_thread::sleep_until(tp)`; // Блокировать поток до момента времени
`this_thread::sleep_until(chrono::system_clock::now()+ chrono::seconds(10))`;

Получить число потоков, которые могут выполняться одновременно на текущем процессоре, можно с помощью статической функции класса `thread`:
`unsigned int thread::hardware_concurrency()`

Поток создается как объект класса *thread* (пространство имен *std*).

```

void myfun(string str) // Потоковая функция может иметь параметры при необходимости
{
    for (int i = 0; i < 10; i++) // 10 раз печатаем
        cout << endl << str.data();
}

int main(int argc, char* argv[])
{
    thread th1(myfun, "Java"), th2(myfun, "C++"); // Создание потоков
    // первый параметр - указатель на потоковую функцию,
    // следующие параметры передаются в потоковую функцию при необходимости

    th1.join(); // Ждем завершение дочернего потока
    th2.join(); // Ждем завершение дочернего потока
    system("pause"); // Останавливаем программу до нажатия любой клавиши
    return 0;
}

```

Альтернативой метода `join` является вызов метода `detach` (отсоединить поток от родительского потока), например, `th1.detach()`; Отсоединенные потоки фактически выполняются в фоновом режиме, владение и управление ими передаются в библиотеку среды выполнения C++, которая гарантирует правильное высвобождение ресурсов, связанных с потоком, при выходе из него. Если не вызвать `join` или `detach`, то при удалении созданных потоков, например, при выходе из блока, в котором они созданы, будет выдано исключение.

14.4 Синхронизация

Проблема синхронизации может возникнуть, когда 2 или более потока пытаются «одновременно» получить доступ к одним и тем же данным (или одному объекту). Также часто один поток должен ожидать сигнала от другого потока, что некоторое событие произошло (например, подготовлены требуемые исходные данные) или потоки должны выполняться в определенном порядке.

Состояние гонки (race condition) — ошибка проектирования многопоточной системы, при которой работа системы зависит от того, в каком порядке выполняются части кода.

Для синхронизации можно использовать объект класса *mutex*.

```
mutex mut; // объект для синхронизации
void myfun(string str) // Потокосовая функция
{
    for (int i = 0; i < 10; i++) // 10 раз печатаем
    {
        mut.lock(); // Блокируем объект
        cout << endl << str.data();
        mut.unlock(); // Снимаем блокировку
    }
}
```

Блокировки

```
std::lock_guard <std::mutex> lock(mut);
(Другой класс unique_lock<> позволяет иметь заблокированный и разблокированный mutex,
lock_guard всегда блокирует mutex).
```

Когда объект перестает существовать блокировка снимается. Принцип:

Resource Acquisition Is Initialization (RAII) – «Получение ресурса есть инициализация»

Программная идиома объектно-ориентированного программирования, смысл которой заключается в том, что с помощью тех или иных программных механизмов получение некоторого ресурса неразрывно совмещается с инициализацией, а освобождение — с уничтожением объекта.

Пример:

```
#include <iostream>
#include <thread>
#include <stdlib.h>
#include <mutex>
using namespace std;
std::mutex mut;
void mythread(std::string str)
{
    for (int i = 0; i < 10; i++)
    {
        mut.lock();
        std::cout << std::endl << str.data();
        mut.unlock();
    }
}
void mythread2()
{
    for (int i = 0; i < 10; i++)
    {
        // mut.lock();
    }
}
```

```

        std::lock_guard<std::mutex> lock(mut); // Здесь снимать блокировку с mut не надо,
она автоматически снимается когда объект перестает существовать
        std::cout << std::endl << "Goodbye Thread";
        // mut.unlock();
    } // Объект lock перестает существовать при выходе из блока mut автоматически
разблокируется
}

void main()
{
    std::cout << "\nN=" << std::thread::hardware_concurrency(); // Число потоков могут
одновременно выполняться
    thread th1(mythread, "Java"), th2(mythread, "C++"); // первый параметр - указатель на
потоковую функцию,
// следующие параметры передаются в потоковую функцию при необходимости
    std::thread::id myid = th1.get_id(); // Идентификатор потока
    mut.lock();
    std::cout << "\nid=" << myid;
    mut.unlock();

    thread th3(mythread2);
    // первый параметр - указатель на потоковую функцию,
    // следующие параметры передаются в потоковую функцию при необходимости

    th1.join(); // Ждем завершение дочернего потока
    th2.join(); // Ждем завершение дочернего потока
    th3.join();

    system("pause");
}

```

Условные переменные

Используется заголовочный файл `<condition_variable>`

Условная переменная это переменная, с помощью которой поток может активировать один или несколько других ожидающих потоков.

Для использования необходимо включить в программу файлы `<mutex>` и `<condition_variable>`. Объявить mutex и условную переменную:

```

mutex mut1;
condition_variable con_var;

```

Поток сигнализирующий о выполнении условия должен выполнить вызов:

```

con_var.notify_one(); или
con_var.notify_all();

```

Любой поток ожидающий выполнение условие должен выполнить вызов:

```

unique_lock<mutex> lock(mut1);
con_var.wait(lock);

```

Пример:

```

#include <future>
#include <mutex>
#include <condition_variable>
#include <iostream>
using namespace std;
mutex mut1;
condition_variable con_var;
bool flag = false; // Флаг для предотвращения ложных срабатываний

```

```

void thread1()
{
    // Имитируем подготовку исходных данных
    cout << "Start Thread1" << endl;
    for (int i = 0; i < 10; i++)
        cout << "Thread 1 i=" << i << endl;
    flag = true; // Данные готовы
    con_var.notify_one(); // Данные готовы (посылаем "сигнал")
}

void thread2()
{
    // Имитируем ожидание данных от первого потока
    cout << "Start Thread2" << endl;
    unique_lock<mutex> lock(mut1);
    while(!flag) con_var.wait(lock); // Ждем "сигнала"
    flag = false; // Условная переменная готова для следующего использования
    cout << "Thread 2 Data read" << endl;
}

void main()
{
    auto f1 = async(launch::async, thread1);
    auto f2 = async(launch::async, thread2);
    system("pause");
}

```

Атомарные операции

Не могут быть прерваны другим потоком, выполняются без перерывов. `atomic` – шаблон (заголовочный файл `atomic`).

Определены следующие классы для стандартных типов:

<code>std::atomic_char</code>	<code>std::atomic<char></code>
<code>std::atomic_schar</code>	<code>std::atomic<signed char></code>
<code>std::atomic_uchar</code>	<code>std::atomic<unsigned char></code>
<code>std::atomic_short</code>	<code>std::atomic<short></code>
<code>std::atomic_ushort</code>	<code>std::atomic<unsigned short></code>
<code>std::atomic_int</code>	<code>std::atomic<int></code>
<code>std::atomic_uint</code>	<code>std::atomic<unsigned int></code>
<code>std::atomic_long</code>	<code>std::atomic<long></code>
<code>std::atomic_ulong</code>	<code>std::atomic<unsigned long></code>
<code>std::atomic_llong</code>	<code>std::atomic<long long></code>
<code>std::atomic_ullong</code>	<code>std::atomic<unsigned long long></code>

Создается атомарный объект (заголовочный файл `<atomic>`): основные операции `store` – присвоить новое значение и `load` – возвращает текущее значение, `fetch_add` – увеличить значение объекта на заданное значение. Ниже в примере несколько потоков работают с обычной переменной, увеличивая ее на 1 (10 потоков каждый по 10000 выполняет инкремент переменной). При этом, каждый раз получаем разный результат, как правило, не 100000.

Пример с обычной переменной:

```
#include <iostream>
#include <future>
#include <vector>
using namespace std;
int value = 0;
void do_work()
{
    for (int i = 0; i < 10000; i++)
        ++value; // Увеличиваем на 1
}

int main()
{
    {
        vector<future<void>> vecFuture;
        for (int i = 0; i < 10; i++) vecFuture.push_back(async(do_work));
    } // При выходе из блока ждем завершения потоков
    std::cout << "Result : " << value << '\n';
}
```

Ниже в примере, выполняется то же самое с атомарным объектом, результат будет при каждом старте один и тот же 100000.

Пример с атомарным объектом:

```
#include <iostream>
#include <future>
#include <vector>
#include <atomic>
using namespace std;
atomic<int> value = 0;
void do_work()
{
    for (int i = 0; i < 10000; i++)
        value.fetch_add(1); // Увеличиваем на 1
}

int main()
{
    {
        vector<future<void>> vecFuture;
        for (int i = 0; i < 10; i++) vecFuture.push_back(async(do_work));
    } // При выходе из блока ждем завершения потоков
    std::cout << "Result : " << value << '\n';
}
```

Далее материал факультативный, посвящен программированию многопоточных программ с использованием платформенно-зависимых средств ОС Windows.

14.5. Создание потока с помощью API – функций Windows

Основные функции:

HANDLE CreateThread(// Создать поток
LPSECURITY_ATTRIBUTES lpThreadAttributes, // дескриптор защиты

```

    SIZE_T dwStackSize,           // начальный размер стека
    LPTHREAD_START_ROUTINE lpStartAddress, // функция потока
    LPVOID lpParameter,          // параметр потока
    DWORD dwCreationFlags,       // опции создания
    LPDWORD lpThreadId           // идентификатор потока
);

```

```

DWORD SuspendThread( // Приостановить поток
HANDLE hThread // дескриптор (хэндел) потока
);

```

```

DWORD ResumeThread( // Возобновить поток
HANDLE hThread // дескриптор (хэндел) потока
);

```

Потоковая функция должна иметь следующий заголовок:
 DWORD WINAPI PotokFun(LPVOID param)

Установить приоритет потока:

```

BOOL SetThreadPriority(
HANDLE hThread, // дескриптор потока
int nPriority // уровень приоритета потока
);

```

Возможные значения уровней приоритета:

```

THREAD_PRIORITY_LOWEST          THREAD_PRIORITY_BELOW_NORMAL
THREAD_PRIORITY_NORMAL          THREAD_PRIORITY_HIGHEST
THREAD_PRIORITY_ABOVE_NORMAL    THREAD_PRIORITY_ERROR_RETURN
THREAD_PRIORITY_TIME_CRITICAL    THREAD_PRIORITY_IDLE

```

Нормальное завершение потока - естественный выход из потоковой функции.

Поток можно завершить из вне с помощью вызова функции:

```

BOOL TerminateThread(
HANDLE hThread, // дескриптор потока
DWORD dwExitCode // код завершения для потока
);

```

Такое завершение потока небезопасное (нет гарантии, что внешние ресурсы, используемые потоком будут освобождены).

Пример:

```

#include <Windows.h>
#include <iostream>
using namespace std;
DWORD WINAPI MyThread1(LPVOID par) // Потоковая функция
{
    char Str[64];

```

```

        strcpy_s(Str, (char *)par); // Копируем строку

        for (int i = 0; i<10; i++)
        {
            cout << endl << Str;

            Sleep(10);
        }

        return 1;
    }

int main(int argc, char* argv[])
{
    HANDLE h1 = CreateThread(0, 0, MyThread1, "C++", 0, 0); // Создаем 1-й поток

    HANDLE h2 = CreateThread(0, 0, MyThread1, "Java", 0, 0); // Создаем 2-й поток
    //      SetThreadPriority(h2, THREAD_PRIORITY_TIME_CRITICAL);

    for (int i = 0; i<10; i++)
        cout << endl << "Main Thread";

    system("pause");
    return 0;
}

```

14.6. Синхронизация потоков

Объекты синхронизации (Windows):

Синхронизирующий объект	Назначение	Используемые функции Win32 API
Взаимное исключение	Запрет доступа более чем одному потоку к общим ресурсам	CreateMutex WaitForSingleObject WaitForMultipleObjects ReleaseMutex CloseHandle
Критическая секция	Запрет доступа более чем одному потоку к одному фрагменту кода	InitializeCriticalSection EnterCriticalSection LeaveCriticalSection DeleteCriticalSection
Семафор	Ограничение числа потоков, имеющих одновременный доступ к общим ресурсам	CreateSemaphore WaitForSingleObject WaitForMultipleObjects ReleaseSemaphore CloseHandle
Событие	Позволяет потоку передавать сигналы другим потокам	CreateEvent SetEvent PulseEvent ResetEvent WaitForSingleObject WaitForMultipleObjects CloseHandle

Использование объектов:

1. *Взаимное исключение*

```
HANDLE HMutex; // Объявляется глобально и все потоки имеют доступ
HMutex=CreateMutex(
NULL, // атрибуты прав доступа по умолчанию
FALSE, // Взаимное исключение изначально свободно
NULL); // не присваивается имя взаимному исключению
```

Работа с синхронизированным объектом

```
WaitForSingleObject(HMutex, // Занять взаимное исключение
INFINITE); // ждите сколько нужно
Count++; // Работа с синхронизированным объектом
ReleaseMutex(HMutex); // Освободить взаимное исключение
```

Удалить взаимное исключение:

```
CloseHandle(HMutex);
```

Пример:

```
#include <windows.h>
#include <iostream>

using namespace std;

HANDLE HMutex; // Объект - взаимное исключение

DWORD WINAPI PotokFun(LPVOID param) // Потоквая функция
{
```

```

char *str = (char *)param;

for (int i = 0; i<10; i++)
{
    WaitForSingleObject(HMutex, // Занять взаимное исключение
        INFINITE); // ждите сколько нужно

    cout << "\n\" << str; // Печатаем строку в кавычках
    //Sleep(10);
    cout << "\""; // Кавычка закрывается
    ReleaseMutex(HMutex); // Кавычка закрывается
}

return 1;
}

```

```

int main(int argc, char* argv[])
{
    DWORD id;
    HMutex = CreateMutex(
        0, // атрибуты прав доступа по умолчанию
        FALSE, // Взаимное исключение изначально свободно
        NULL); // не присваивается имя взаимному исключению

    CreateThread(0, 0, PotokFun, "C++", 0, &id);
    CreateThread(0, 0, PotokFun, "Java", 0, &id);

    system("pause");
    CloseHandle(HMutex);

    return 0;
}

```

2. Семафор

Работает по аналогии с взаимным исключением только доступ к объекту может получить не один поток, а их число определяется параметром `MaximumCount` (при занятии потоком семафора текущее значение счетчика уменьшается на 1, семафор полностью занят если значение счетчика равно 0).

```

HANDLE CreateSemaphore( // Создать семафор
    LPSECURITY_ATTRIBUTES lpAttr, // Атрибуты доступа
    LONG InitialCount, // Начальное значение семафора
    LONG MaximumCount, // Макс. значение
    LPCTSTR lpName ); // Имя семафора

```

Возможная схема использования:

```

WaitForSingleObject(HSem, // Занять семафор
    INFINITE); // ждите сколько нужно
Count++; // Работа с синхронизированным объектом
ReleaseSemaphore(HSem, 1, 0); // Освободить одно место семафора

```

Пример:

```

#include <windows.h>
#include <iostream>

```

```

using namespace std;
HANDLE hSem; // Объект - семафор
DWORD WINAPI PotokFun(LPVOID param)
{
    char *str = (char *)param;

    for (int i = 0; i<10; i++)
    {
        WaitForSingleObject(hSem, // Занять одно место в семафоре
            INFINITE); // ждите сколько нужно
        cout<<"\n\"<< str; // Печатаем строку в кавычках
        // Sleep(10); // Задержка в мс
        cout<<"\""; // Кавычка закрывается
        ReleaseSemaphore(hSem, 1, 0); // Освободить одно место в семафоре
    }

    return 1;
}
int main(int argc, char* argv[])
{
    hSem = CreateSemaphore(0, 1, 1, 0);
    CreateThread(0, 0, PotokFun, "C++", 0, 0);
    CreateThread(0, 0, PotokFun, "Java", 0, 0);
    system("pause");
    CloseHandle(hSem);
    return 0;
}

```

3. *Критическая секция* - это фрагмент программы, защищенный от одновременного выполнения несколькими потоками. Критическую секцию в данный момент может выполнять только один поток.

InitializeCriticalSection - данная функция *создает объект под названием критическая секция*. Параметры функции:

Указатель на структуру, CRITICAL_SECTION. Поля данной структуры используются только внутренними процедурами, их смысл безразличен.

EnterCriticalSection - *войти в критическую секцию*. После выполнения этой функции данный поток становится владельцем данной секции. Следующий поток, вызвав данную функцию, будет находиться в состоянии ожидания. Параметр функции такой же, что и в предыдущей функции.

LeaveCriticalSection - *покинуть критическую секцию*. После этого второй поток, который был остановлен функцией **EnterCriticalSection**, станет владельцем критической секции. Параметр функции **LeaveCriticalSection** такой же, как и у предыдущих функций.

DeleteCriticalSection - *удалить объект "критическая секция"*. Параметр аналогичен предыдущим.

Пример:

```

#include <Windows.h>
#include <iostream>
using namespace std;
CRITICAL_SECTION cs;
DWORD WINAPI PotokFun(LPVOID param)
{
    char *str = (char *)param;

    EnterCriticalSection(&cs);
    for (int i = 0; i<10; i++)
    {

```

```

        cout<<"\n\"<< str; // Печатаем строку в кавычках
        Sleep(10); // Задержка в мс
        cout<<"\""; // Кавычка закрывается
    }
    LeaveCriticalSection(&cs);

    return 1;
}
int main(int argc, char* argv[])
{
    InitializeCriticalSection(&cs);
    CreateThread(0, 0, PotokFun, "C++", 0, 0);
    CreateThread(0, 0, PotokFun, "Java", 0, 0);
    system("pause");
    DeleteCriticalSection(&cs);
    return 0;
}

```

4. Событие

```

HANDLE hEvent;
hEvent=CreateEvent(0,
    false, // TRUE - событие со сбросом вручную FALSE — событие с автосбросом
    false, // событие вкл. (произошло) – 1,
           // событие выкл. (не произошло) - 0.
    0);
WaitForSingleObject(hEvent,
    INFINITE);

```

// Программа далее не выполняется

Когда событие произошло необходимо вызвать в другом потоке функцию

```
SetEvent(hEvent);
```

Для сброса события используется функция ResetEvent.

Пример:

```

#include <Windows.h>
#include <iostream>
using namespace std;
HANDLE hEvent;

DWORD WINAPI PotokFun(LPVOID param)
{
    char *str = (char *)param;
    for (int i = 0; i<20; i++)
    {
        cout << "\n\"<< str; // Печатаем строку в кавычках
        //Sleep(10);
        cout << "\""; // Кавычка закрывается
    }
    SetEvent(hEvent); // Событие произошло
    return 1;
}

int main(int argc, char* argv[])
{
    hEvent = CreateEvent(0,
        false, // TRUE событие со сбросом вручную FALSE – событие с автосбросом
        false, // свободное (TRUE) занятое (FALSE).

```

```

        0);
    CreateThread(0, 0, PotokFun, "C++", 0, 0);
    WaitForSingleObject(hEvent, // Ожидаем наступления события
        INFINITE);
    cout<<"\nEnd Main";
    system("pause");
    CloseHandle(hEvent);

    return 0;
}

```

Синхронизация разных процессов

С помощью объектов синхронизации можно организовать синхронизацию различных приложений (процессов). Например, доступ к объекту синхронизации, созданному в одном приложении, можно получить в другом приложении:

```

HANDLE OpenEvent(
    DWORD dwDesiredAccess, // флаги доступа
    BOOL bInheritHandle, // режим наследования
    LPCTSTR lpName // имя события
);

```

Возможные значения флагов доступа:

- EVENT_ALL_ACCESS
- EVENT_MODIFY_STATE
- SYNCHRONIZE

14.7. Пул потоков (факультативно)

Чтобы не создавать и удалять потоки, можно создать множество потоков или пул (они могут находиться в состоянии ожидания). Пулу потоков передаются задания для решения, представляющие собой потоковые функции.

Основные функции:

Создать пул:

```
PTP_POOL WINAPI CreateThreadpool(PVOID reserved);
```

Установить минимальное и максимальное число потоков в пуле:

```
BOOL WINAPI SetThreadpoolThreadMinimum(PTP_POOL ptp,
    DWORD cthrdMin);
```

```
VOID WINAPI SetThreadpoolThreadMaximum(PTP_POOL ptp,
    DWORD cthrdMost);
```

Инициализировать среду:

```
void InitializeThreadpoolEnvironment(
    PTP_CALLBACK_ENVIRON pcbe);
```

Связать пул потоков со средой:

```
void SetThreadpoolCallbackPool(
    PTP_CALLBACK_ENVIRON pcbe,
```

PTP_POOL ptp);

Создать работу для пула(связать с функцией):

PTP_WORK WINAPI CreateThreadpoolWork(PTP_WORK_CALLBACK pfnwk, PVOID Context,
PTP_CALLBACK_ENVIRON pcbe);

Передать работу на выполнение:

VOID WINAPI SubmitThreadpoolWork(PTP_WORK pwk);

Функция для закрытия объектов

- CloseThreadpoolWork
- DestroyThreadpoolEnvironment
- CloseThreadpool

Пример:

```
#include <Windows.h>
#include <stdio.h>
HANDLE HMutex; // Взаимное исключение для синхронизации
struct PARAM // Данные для выполнения задачи
{
    double x, y;
};
int Num1 = 1;
VOID CALLBACK sum(PTP_CALLBACK_INSTANCE Instance, // Функция для выполнения суммы (работа для
пула потоков)
    PVOID p,
    PTP_WORK Work)
{
    PARAM *pPar = (PARAM *)p; // Копируем указатель на данные
    double rez = pPar->x + pPar->y;

    WaitForSingleObject(HMutex, // Занять взаимное исключение
        INFINITE); // ждите сколько нужно

    printf("\n\"Num=%d %.3f=%.2f+%.2f\"", Num1, rez, pPar->x, pPar->y);
    Sleep(10);
    pPar->x++; pPar->y++; Num1++;
    printf("\");
    ReleaseMutex(HMutex);
}

VOID CALLBACK proiz(PTP_CALLBACK_INSTANCE Instance, // Функция для выполнения произведения
(работа для пула потоков)
    PVOID p,
    PTP_WORK Work)
{
    PARAM *pPar = (PARAM *)p; // Копируем указатель на данные
    double rez = pPar->x*pPar->y;

    WaitForSingleObject(HMutex, // Занять взаимное исключение
        INFINITE); // ждите сколько нужно
```

```

    printf("\n\"Num=%d %.3f=%.2f*%.2f", Num1, rez, pPar->x, pPar->y);
    Sleep(10);
    pPar->x++; pPar->y++; Num1++;
    printf("\");
    ReleaseMutex(HMutex);
}

VOID CALLBACK div(PTP_CALLBACK_INSTANCE Instance, // Функция для выполнения деления (работа для
пула потоков)
    PVOID p,
    PTP_WORK Work)
{
    PARAM *pPar = (PARAM *)p; // Копируем указатель на данные
    double rez = pPar->x / pPar->y;

    WaitForSingleObject(HMutex, // Занять взаимное исключение
        INFINITE); // ждите сколько нужно

    printf("\n\"Num=%d %.3f=%.2f/%.2f", Num1, rez, pPar->x, pPar->y);
    Sleep(10);
    pPar->x++; pPar->y++; Num1++;
    printf("\");
    ReleaseMutex(HMutex);
}

VOID CALLBACK minus(PTP_CALLBACK_INSTANCE Instance, // Функция для выполнения вычитания (работа
для пула потоков)
    PVOID p,
    PTP_WORK Work)
{
    PARAM *pPar = (PARAM *)p; // Копируем указатель на данные
    double rez = pPar->x - pPar->y;

    WaitForSingleObject(HMutex, // Занять взаимное исключение
        INFINITE); // ждите сколько нужно

    printf("\n\"Num=%d %.3f=%.2f-%.2f", Num1, rez, pPar->x, pPar->y);
    Sleep(10);
    pPar->x++; pPar->y++; Num1++;
    printf("\");
    ReleaseMutex(HMutex);
}

int main(int argc, char* argv[])
{
    HMutex = CreateMutex(
        0, // атрибуты прав доступа по умолчанию
        FALSE, // Взаимное исключение изначально свободно
        NULL); // не присваивается имя взаимному исключению
    PARAM Par;
    PTP_POOL pt = CreateThreadpool(NULL); // Создать пул

    if (pt == NULL)
        printf("Error!!!");
    else printf("Ok!");
}

```

```

// Установить минимальное и максимальное число потоков в пуле
SetThreadPoolThreadMinimum(pt,
    2);
SetThreadPoolThreadMaximum(pt,
    4);

TP_CALLBACK_ENVIRON enf;
InitializeThreadPoolEnvironment(&enf); // Инициализировать среду
SetThreadPoolCallbackPool(&enf,
    pt); // Связать пул потоков со средой
Par.x = 1; // Данные
Par.y = 1;
// Создать работы для пула (связать с функцией обратного вызова)
PTP_WORK work1 = CreateThreadPoolWork((PTP_WORK_CALLBACK)sum, // Сумма
    &Par,
    &enf);
PTP_WORK work2 = CreateThreadPoolWork((PTP_WORK_CALLBACK)div, // Деление
    &Par,
    &enf);
PTP_WORK work3 = CreateThreadPoolWork((PTP_WORK_CALLBACK)proiz, // Произведение
    &Par,
    &enf);
PTP_WORK work4 = CreateThreadPoolWork((PTP_WORK_CALLBACK)minus, // Вычитание
    &Par,
    &enf);

for (int i = 0; i<50; i++) // Передать работу на выполнение в цикле
{
    SubmitThreadPoolWork(work1);
    SubmitThreadPoolWork(work2);
    SubmitThreadPoolWork(work3);
    SubmitThreadPoolWork(work4);

}
system("pause");
// Далее закрытие объектов
CloseThreadPoolWork(work1);
CloseThreadPoolWork(work2);
CloseThreadPoolWork(work3);
CloseThreadPoolWork(work4);
DestroyThreadPoolEnvironment(&enf);
CloseThreadPool(pt);
CloseHandle(HMutex);

return 0;
}

```